APACHE HADOOP MAPREDUCE

Name: Nanditha M

USN: 4GW23CI033

Semester: 7th sem

Aim:

The aim of this project is to develop a Banking Transaction Analysis system using Hadoop MapReduce to calculate the total transaction amount for each customer. The system uses Python and the MRJob framework to implement the MapReduce programming model and Hadoop HDFS to store and process transaction data efficiently.

Problem Statement:

Banks generate a large volume of transaction records containing customer IDs and transaction amounts. Analyzing these records manually to determine the total transaction amount for each customer can be time-consuming and inefficient, especially when the dataset is large.

This project addresses the problem by implementing a Hadoop MapReduce-based solution that processes transaction records and calculates the total transaction amount for each customer. The Mapper extracts the customer ID and transaction amount, Hadoop performs the shuffle and sort operation to group transactions belonging to the same customer, and the Reducer calculates the total amount for each customer.

For example, given transaction records such as:

C001,5000
C002,3000
C001,2000
C003,7000
C002,1500
C001,1000

The system produces:

C001 8000
C002 4500
C003 7000

Thus, the system provides an efficient way to analyze and aggregate large-scale banking transaction data using the MapReduce programming model.

Tools and Technologies Used:

- **Python 3**: Used to develop the MapReduce program

- **MRJob**: A Python framework used to implement the Mapper and Reducer functions.
- **Apache Hadoop**: Used as the distributed data processing framework.
- **Hadoop HDFS**: Used to store the banking transaction input file.
- **Hadoop MapReduce**: Used to process and aggregate transaction data.
- **Linux/Ubuntu Terminal**: Used to create files, execute commands, and run the application.
- **CSV**: Used as the format for storing transaction records.

Working of the MapReduce Program

Mapper Function

- The mapper() function reads each transaction record and separates it into fields. The first field represents the customer ID, while the second field represents the transaction amount.
- The Mapper outputs the customer ID as the key and the transaction amount as the value.

Shuffle and Sort

- After the Mapper generates key-value pairs, Hadoop automatically performs the Shuffle and Sort phase.
- During this phase, all transaction amounts belonging to the same customer ID are grouped together.

Reducer Function

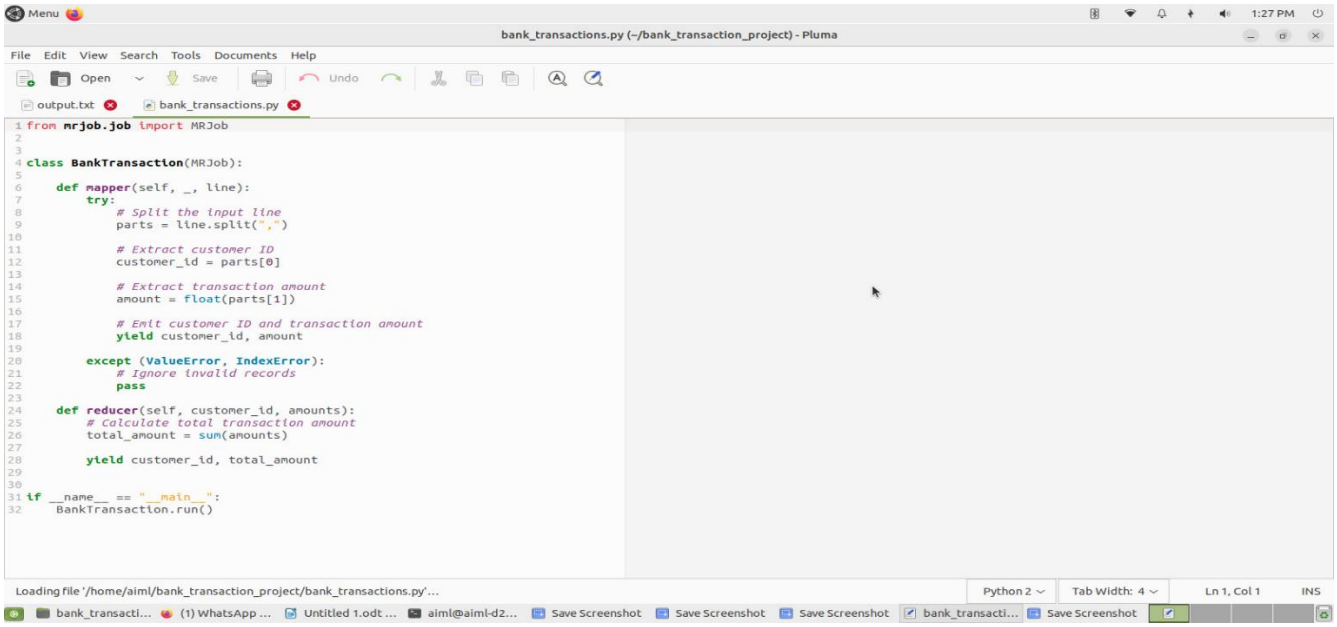
- The reducer() function receives one customer ID along with the list of all transaction amounts associated with that customer.
- It calculates the sum of these transaction amounts and produces the total transaction amount for the customer.

Execution Steps

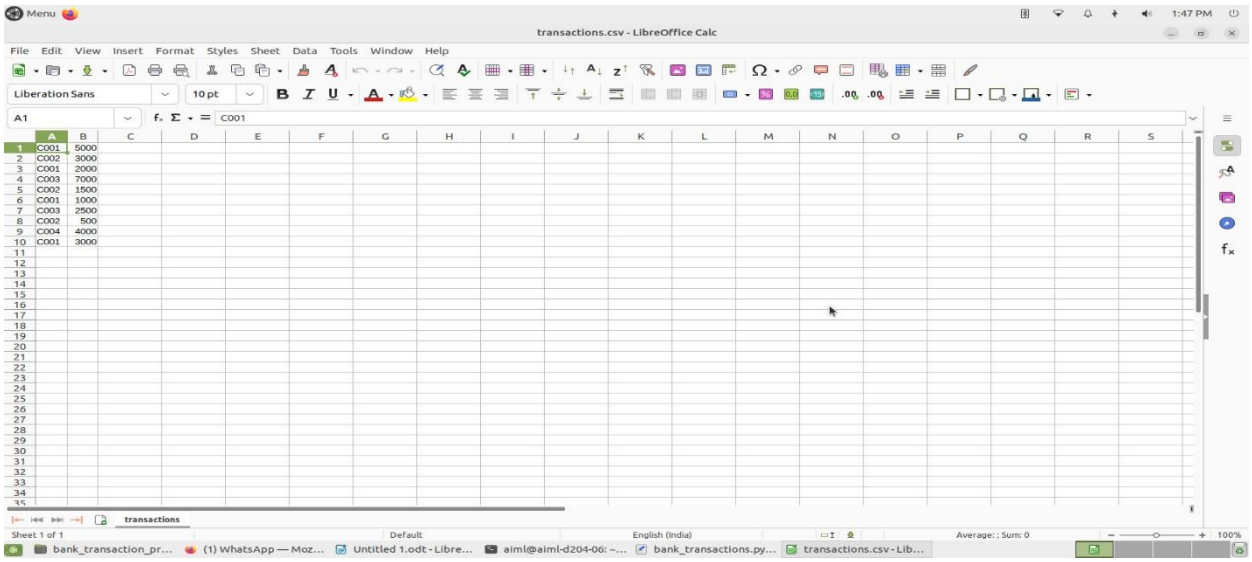
- **mkdir -p ~/bank_transaction_**project – Creates a new project directory named **bank_transaction_project**.
- **cd ~/bank_transaction_project** – Navigates to the created project directory.
- **nano transactions.csv** – Creates and edits the **transactions.csv** file containing customer transaction records.
- **cat transactions.csv** – Displays the contents of the transaction CSV file.
- **nano bank_transactions.py** – Creates and edits the Python program implementing the MapReduce logic.
- **cat bank_transactions.py** – Displays the contents of the Python MapReduce program.
- **python3 --version** – Checks the installed version of Python 3.

- **pip3 install mrjob** – Installs the **MRJob** Python framework required to implement and execute the MapReduce program.
- **python3 bank_transactions.py transactions.csv** – Executes the MapReduce program in **local mode** using the transaction CSV file.
- **hdfs dfs -mkdir -p /bank_transactions/input** – Creates an input directory in **Hadoop HDFS** for storing the transaction data.
- **hdfs dfs -put transactions.csv /bank_transactions/input/** – Uploads the local transaction CSV file to the HDFS input directory.
- **hdfs dfs -ls /bank_transactions/input** – Displays the files available in the HDFS input directory and verifies that the transaction file was uploaded successfully.
- **python3 bank_transactions.py -r hadoop** hdfs:///bank_transactions/input/transactions.csv --output-dir hdfs:///bank_transactions/output – Executes the MapReduce program on the **Hadoop cluster** using the transaction file stored in HDFS and stores the processed results in the HDFS output directory.
- **hdfs dfs -ls /bank_transactions/output** – Lists the output files generated by the Hadoop MapReduce job.
- **hdfs dfs -cat /bank_transactions/output/part-*** – Displays the final MapReduce results stored in the Hadoop output files.

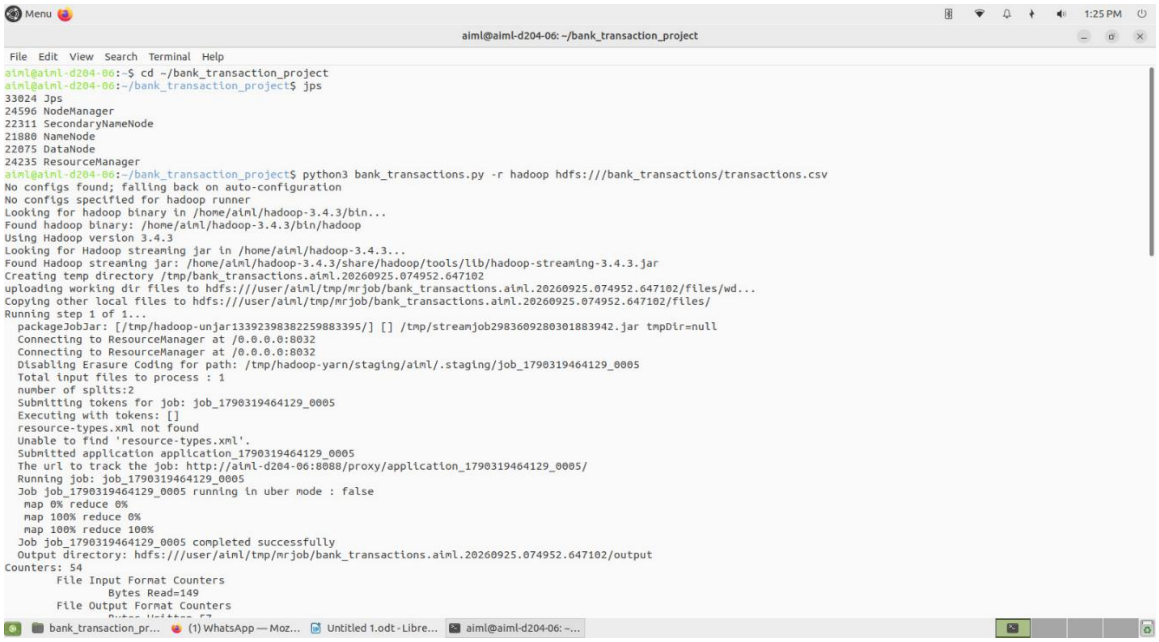
File name: bank_transactions.py

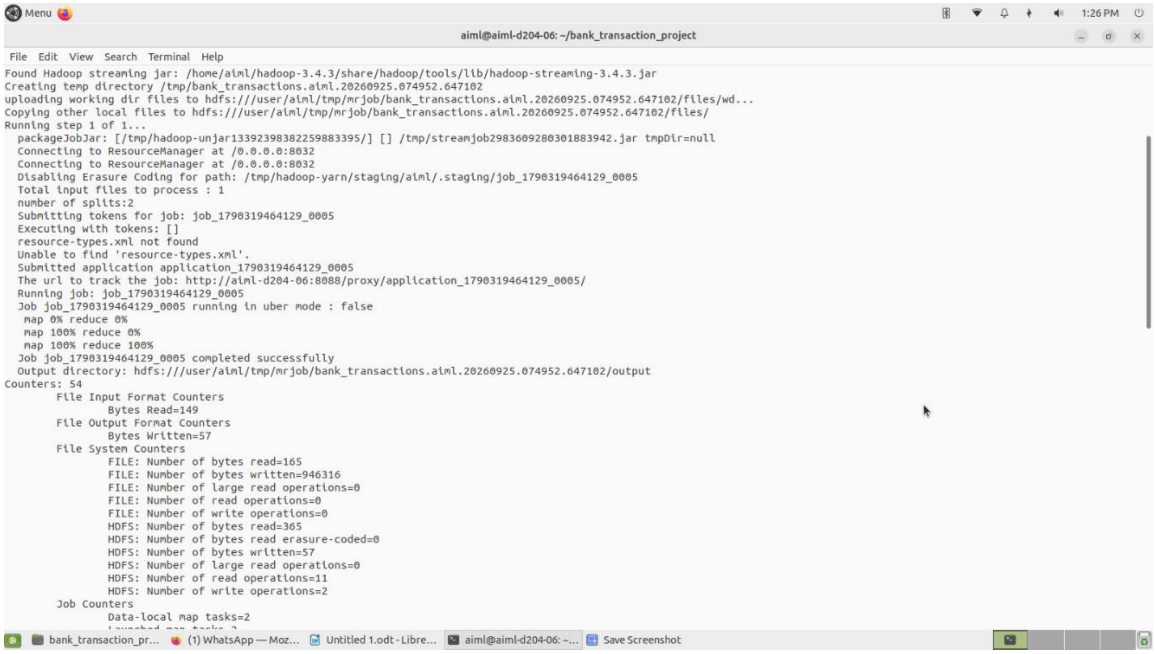


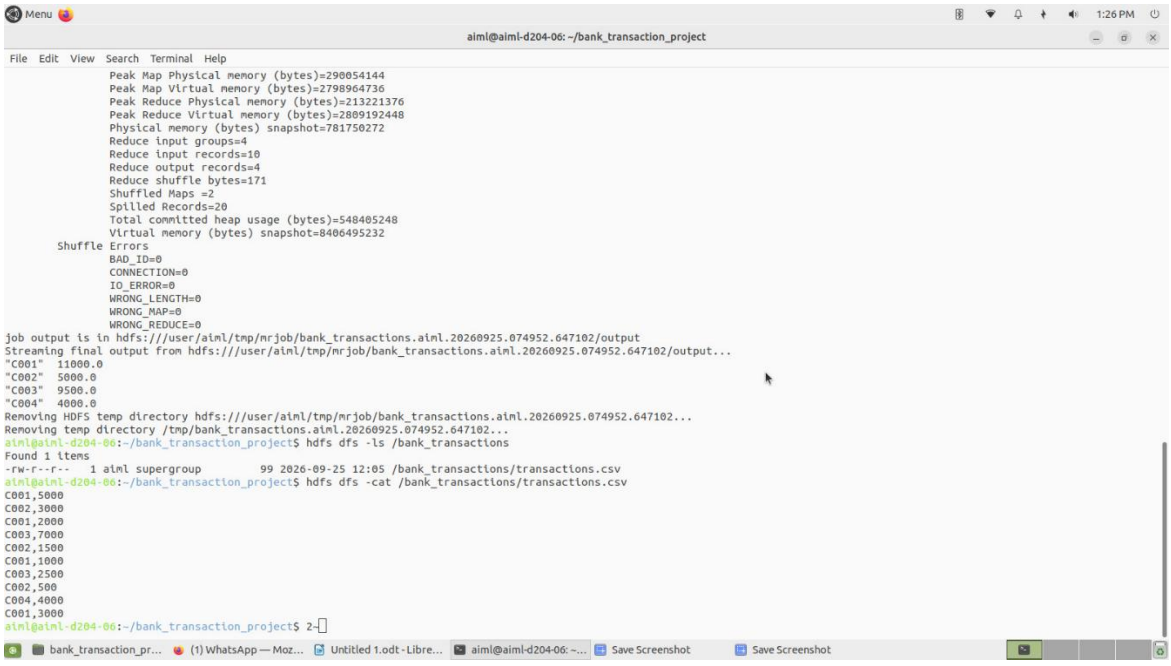
Transactions.csv

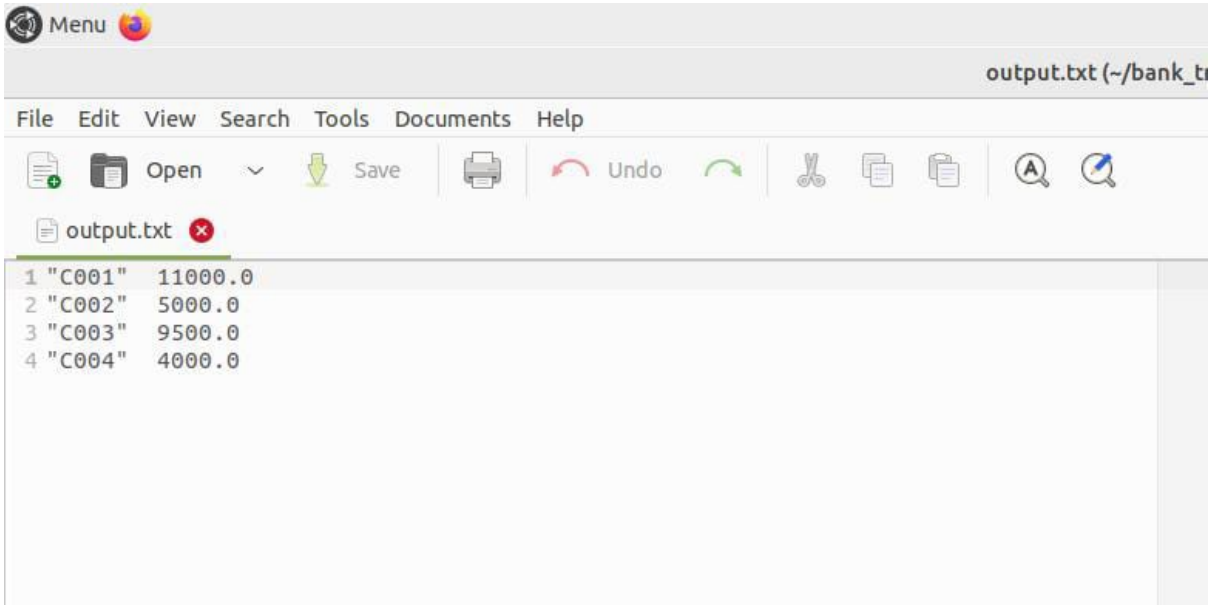

transactions.csv - LibreOffice Calc
File Edit View Insert Format Styles Sheet Data Tools Window Help
Liberation Sans
10 pt
A1
C001
transactions
Sheet 1 of 1
Default
English (India)
Average: ; Sum: 0
100%

Output


aiml@aiml-d204-06: ~/bank_transaction_project
File Edit View Search Terminal Help
aiml@aiml-d204-06:~$ cd ~/bank_transaction_project
aiml@aiml-d204-06:~/bank_transaction_project$ jps
33024 Jps
24596 NodeManager
22311 SecondaryNameNode
21888 NameNode
22075 DataNode
24235 ResourceManager
aiml@aiml-d204-06:~/bank_transaction_project$ python3 bank_transactions.py -r hadoop hdfs:///bank_transactions/transactions.csv
No configs found; falling back on auto-configuration
No configs specified for hadoop runner
Looking for hadoop binary in /home/aiml/hadoop-3.4.3/bin...
Found hadoop binary: /home/aiml/hadoop-3.4.3/bin/hadoop
Using Hadoop version 3.4.3
Looking for Hadoop streaming jar in /home/aiml/hadoop-3.4.3...
Found Hadoop streaming jar: /home/aiml/hadoop-3.4.3/share/hadoop/tools/lib/hadoop-streaming-3.4.3.jar
Creating temp directory /tmp/bank_transactions.aiml.20260925.074952.647102
uploading working dir files to hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/files/wd...
Copying other local files to hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/files/
Running step 1 of 1...
packageJobJar: [/tmp/hadoop-unjar1339239838225988 3395/] [] /tmp/streamjob2983609280301883942.jar tmpDir=null
Connecting to ResourceManager at /0.0.0.0:8032
Connecting to ResourceManager at /0.0.0.0:8032
Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aiml/.staging/job_1790319464129_0005
Total input files to process : 1
number of splits:2
Submitting tokens for job: job_1790319464129_0005
Executing with tokens: []
resource-types.xml not found
Unable to find 'resource-types.xml'.
Submitted application application_1790319464129_0005
The url to track the job: http://aiml-d204-06:8088/proxy/application_1790319464129_0005/
Running job: job_1790319464129_0005
Job job_1790319464129_0005 running in uber mode : false
map 0% reduce 0%
map 100% reduce 0%
map 100% reduce 100%
Job job_1790319464129_0005 completed successfully
Output directory: hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/output
Counters: 54
File Input Format Counters
Bytes Read=149
File Output Format Counters


Menu
1:26 PM
aiml@aiml-d204-06: ~/bank_transaction_project
File Edit View Search Terminal Help
Found Hadoop streaming jar: /home/aiml/hadoop-3.4.3/share/hadoop/tools/lib/hadoop-streaming-3.4.3.jar
Creating temp directory /tmp/bank_transactions.aiml.20260925.074952.647102
uploading working dir files to hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/files/wd...
Copying other local files to hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/files/
Running step 1 of 1...
packageJobJar: [/tmp/hadoop-unjar1339239838225988339 5/] [] /tmp/streamjob2983609280301883942.jar tmpDir=null
Connecting to ResourceManager at /0.0.0.0:8032
Connecting to ResourceManager at /0.0.0.0:8032
Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aiml/.staging/job_1790319464129_0005
Total input files to process : 1
number of splits:2
Submitting tokens for job: job_1790319464129_0005
Executing with tokens: []
resource-types.xml not found
Unable to find 'resource-types.xml'.
Submitted application application_1790319464129_0005
The url to track the job: http://aiml-d204-06:8088/proxy/application_1790319464129_0005/
Running job: job_1790319464129_0005
Job job_1790319464129_0005 running in uber mode : false
map 0% reduce 0%
map 100% reduce 0%
map 100% reduce 100%
Job job_1790319464129_0005 completed successfully
Output directory: hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/output
Counters: 54
File Input Format Counters
Bytes Read=149
File Output Format Counters
Bytes Written=57
File System Counters
FILE: Number of bytes read=165
FILE: Number of bytes written=946316
FILE: Number of large read operations=0
FILE: Number of read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=365
HDFS: Number of bytes read erasure-coded=0
HDFS: Number of bytes written=57
HDFS: Number of large read operations=0
HDFS: Number of read operations=11
HDFS: Number of write operations=2
Job Counters
Data-local map tasks=2
bank_transaction_pr...
(1) WhatsApp — Moz...
Untitled 1.odt - Libre...
aiml@aiml-d204-06: ~...
Save Screenshot


Menu
1:26 PM
aiml@aiml-d204-06: ~/bank_transaction_project
File Edit View Search Terminal Help
Peak Map Physical memory (bytes)=290054144
Peak Map Virtual memory (bytes)=2798964736
Peak Reduce Physical memory (bytes)=213221376
Peak Reduce Virtual memory (bytes)=2809192448
Physical memory (bytes) snapshot=781750272
Reduce input groups=4
Reduce input records=10
Reduce output records=4
Reduce shuffle bytes=171
Shuffled Maps =2
Spilled Records=20
Total committed heap usage (bytes)=548405248
Virtual memory (bytes) snapshot=8406495232
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
job output is in hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/output
Streaming final output from hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102/output...
"C001" 11000.0
"C002" 5000.0
"C003" 9500.0
"C004" 4000.0
Removing HDFS temp directory hdfs:///user/aiml/tmp/mrjob/bank_transactions.aiml.20260925.074952.647102...
Removing temp directory /tmp/bank_transactions.aiml.20260925.074952.647102...
aiml@aiml-d204-06:~/bank_transaction_project$ hdfs dfs -ls /bank_transactions
Found 1 items
-rw-r--r-- 1 aiml supergroup 99 2026-09-25 12:05 /bank_transactions/transactions.csv
aiml@aiml-d204-06:~/bank_transaction_project$ hdfs dfs -cat /bank_transactions/transactions.csv
C001,5000
C002,3000
C001,2000
C003,7000
C002,1500
C001,1000
C003,2500
C002,500
C004,4000
C001,3000
aiml@aiml-d204-06:~/bank_transaction_project$ 2-
bank_transaction_pr...
(1) WhatsApp — Moz...
Untitled 1.odt - Libre...
aiml@aiml-d204-06: ~...
Save Screenshot
Save Screenshot



Brief Explanation of Output

The output of the Banking Transaction Analysis using Hadoop MapReduce project provides the total transaction amount for each customer. The Mapper extracts the Customer ID and transaction amount from each transaction record. During the Shuffle and Sort phase, Hadoop groups all transaction amounts associated with the same Customer ID. The Reducer then calculates the sum of these amounts and generates the final output.

The output consists of two fields: Customer ID and Total Transaction Amount.

For example, customer C001 has transactions of 5000, 2000, 1000, and 3000, resulting in a total of 11000.0.

The results confirm that the MapReduce program successfully processes the transaction data, groups records according to Customer ID, and calculates the total transaction amount for each customer. This demonstrates the effective use of Hadoop MapReduce for transaction data aggregation.

Conclusion

The Banking Transaction Analysis using Hadoop MapReduce project was successfully implemented to calculate the total transaction amount for each customer. The transaction data was stored in Hadoop HDFS and processed using a Python-based MapReduce program with the MRJob framework.

The Mapper extracted the customer ID and transaction amount, while the Shuffle and Sort phase grouped transactions based on customer ID. The Reducer then calculated the total transaction amount for each customer.

The project demonstrates how Hadoop MapReduce can be used to efficiently process and analyze large volumes of banking transaction data. The successful execution of the project produced the required customer-wise transaction totals and achieved the main objective of the system.